

Arduino Cookbook : De la Théorie à la Pratique

Chapitre 10 : Contrôle d'appareils externes et Télécommandes

Présenté par : Dr B. DIOP

29 mars 2026

Référence : Michael Margolis, *Arduino Cookbook*, 2nd Edition, O'Reilly

10.0 Introduction : Sortir du monde 5 Volts

Interagir avec le monde réel

L'Arduino fonctionne en basse tension continue (5V) et avec des courants très faibles (40 mA par broche). Pour interagir avec des appareils domestiques, il faut des interfaces d'isolation et de communication.

Dans ce chapitre, nous allons apprendre à :

- Recevoir et décoder les signaux d'une télécommande Infrarouge (IR).
- Émettre des signaux IR pour contrôler une TV ou une chaîne hi-fi.
- Utiliser des Optocoupleurs pour isoler électriquement des circuits.
- Piloter des charges de forte puissance avec des Relais Électromécaniques.
- Découvrir les Relais Statiques (Solid State Relays - SSR).

10.1 La communication Infrarouge (IR)

Qu'est-ce que l'Infrarouge ? C'est une lumière invisible à l'œil humain (longueur d'onde autour de 940 nm).

Le problème de la lumière ambiante : Le soleil et les ampoules émettent aussi de l'infrarouge. Pour éviter que la télécommande ne soit perturbée par la lumière du jour, le signal est **modulé**.

- **La Modulation à 38 kHz :** La LED infrarouge de la télécommande clignote très rapidement à 38 000 fois par seconde pour représenter un "1" logique.
- Le récepteur filtre tout ce qui ne clignote pas à cette fréquence exacte.

10.2 Le Récepteur Infrarouge (TSOP38238)

Composant tout-en-un : Contrairement à une simple photodiode, un récepteur IR comme le TSOP38238 intègre une lentille, un filtre optique, un amplificateur et un démodulateur 38 kHz.

Câblage (3 broches) :

- **OUT (Signal)** : Connecté à une broche numérique (ex : Broche 11).
- **GND** : Masse.
- **VCC** : 5V.

[Insérer le schéma de câblage du récepteur IR]

Code : Décoder un signal de télécommande

La bibliothèque de référence est IRremote.h. Elle décode automatiquement les protocoles complexes (NEC, Sony, RC5).

```
#include <IRremote.h>

const int recvPin = 11;
IRrecv irrecv(recvPin); // Initialisation du recepteur
decode_results results; // Structure pour stocker le code reçu

void setup() {
  Serial.begin(9600);
  irrecv.enableIRIn(); // Demarre le recepteur
}

void loop() {
  if (irrecv.decode(&results)) { // Si un code est reçu
    Serial.print("Code reçu (Hex) : ");
    Serial.println(results.value, HEX); // Affiche le code en Hexadecimal
    irrecv.resume(); // Prepare a recevoir le signal suivant
  }
}
```

10.3 Créer une télécommande (Émetteur IR)

Émettre de l'Infrarouge : Pour contrôler un appareil, l'Arduino doit reproduire exactement la séquence de clignotements 38 kHz et les timings spécifiques à la marque de la TV.

Câblage :

- Une **LED Infrarouge** classique (émettrice).
- Une **résistance série** (ex : 220 Ω pour protéger la broche, ou un montage avec transistor pour une plus longue portée).
- **Important :** Avec IRremote, la broche d'émission est fixée par le matériel (généralement la **broche 3** sur Arduino Uno, car elle utilise le Timer 2).

Code : Envoyer un code IR (Ex : Allumer la TV)

Une fois que vous avez "capturé" le code hexadécimal de votre vraie télécommande avec le montage précédent (ex : 0xF4BA2988 pour le protocole NEC), vous pouvez le renvoyer.

```
#include <IRremote.h>

IRsend irsend; // Utilise implicitement la broche 3 (PWM)

void setup() {
  Serial.begin(9600);
}

void loop() {
  // Envoie le code NEC 0xF4BA2988 (32 bits de longueur)
  irsend.sendNEC(0xF4BA2988, 32);
  Serial.println("Signal envoye !");

  delay(5000); // Attend 5 secondes avant de renvoyer
}
```

10.4 Isoler les circuits : L'Optocoupleur

Principe de l'Isolation Galavanique : Parfois, on veut qu'un Arduino "appuie" sur un bouton d'un autre appareil (ex : déclencheur d'un appareil photo numérique) sans relier électriquement les deux appareils, pour éviter les courts-circuits ou les boucles de masse.

L'Optocoupleur (ex : 4N35) :

- Il contient une **LED** et un **Phototransistor** enfermés dans une puce noire.
- L'Arduino allume la LED interne. La lumière active le transistor qui agit comme un interrupteur pour le circuit externe.
- **Résultat :** Aucune connexion électrique directe (seulement un rayon lumineux).

10.5 Contrôler des charges puissantes (Introduction)

La limite de l'Arduino : 5 Volts DC, 40 Milliampères. **Un appareil domestique (ex : Lampe de salon)** : 220 Volts AC, plusieurs Ampères.

Avertissement de sécurité absolu

Le courant alternatif du secteur (220V/110V) est **mortel**. Ne reliez **JAMAIS** l'Arduino directement au réseau électrique. Tout contrôle doit se faire via des relais correctement dimensionnés et isolés.

10.6 Le Relais Électromécanique

Qu'est-ce qu'un relais ? C'est un interrupteur mécanique actionné par un électro-aimant (une bobine).

Les 3 bornes de la sortie (haute tension) :

- **COM (Commun)** : L'arrivée du courant secteur.
- **NO (Normally Open)** : Contact Normalement Ouvert (le courant ne passe pas au repos).
- **NC (Normally Closed)** : Contact Normalement Fermé (le courant passe au repos).

[Insérer le schéma de fonctionnement d'un relais : Bobine et Contacts]

10.7 Piloter un relais "nu" (sans module)

Un relais nu contient une bobine. Tout comme pour un moteur DC (Chapitre 8), l'Arduino ne peut pas l'alimenter directement.

Le circuit obligatoire comprend :

1. **Un Transistor (NPN)** : Pour amplifier le courant de la broche Arduino et laisser passer l'énergie nécessaire à la bobine.
2. **Une Diode de roue libre (Flyback Diode)** : Montée en parallèle inverse sur la bobine pour absorber le pic de tension lorsque l'électro-aimant se désactive.

Remarque : Les "Modules Relais" tout-faits (sur PCB bleu/rouge) intègrent déjà le transistor, la diode, et souvent un optocoupleur pour la sécurité.

Code : Allumer une lampe avec un Module Relais

Logique Inversée (Active-Low)

Attention : La grande majorité des modules relais pour Arduino s'activent lorsque l'on met la broche à LOW (0V) et se désactivent à HIGH (5V).

```
const int relayPin = 7;

void setup() {
  // Initialiser en HIGH AVANT de déclarer en OUTPUT pour éviter
  // que le relais ne claque brièvement au démarrage
  digitalWrite(relayPin, HIGH);
  pinMode(relayPin, OUTPUT);
}

void loop() {
  digitalWrite(relayPin, LOW); // Active le relais (La lampe s'allume)
  delay(5000);

  digitalWrite(relayPin, HIGH); // Désactive le relais (La lampe s'éteint)
  delay(5000);
}
```

10.8 Les Relais Statiques (Solid State Relays - SSR)

Le successeur du relais mécanique : Le SSR fait la même chose, mais sans aucune pièce mobile. Il utilise des semi-conducteurs de puissance (Triacs).

Avantages du SSR :

- Totalelement silencieux (pas de "clac").
- Durée de vie quasi-infinie (pas d'usure des contacts mécaniques).
- Commutation "Zero-Cross" (s'allume/s'éteint quand l'onde AC passe par 0V, réduisant les parasites).

Inconvénients : Chauffent lors des forts courants (nécessitent un radiateur), plus chers, et généralement conçus uniquement pour du courant alternatif (AC).

10.9 Projet : Thermostat Intelligent (Concept)

En combinant le capteur de température (Chapitre 6) et le Relais (Chapitre 10), on crée un système complet de régulation.

```
const int tempPin = A0;
const int chauffageRelay = 7;
const float TEMP_CIBLE = 22.0;

void loop() {
    float tempActuelle = lireTemperature(); // Fonction personnalisée

    // Hysteresis : On prévoit une marge pour éviter que le relais
    // ne claque sans arrêt autour de la température cible.
    if (tempActuelle < (TEMP_CIBLE - 0.5)) {
        digitalWrite(chauffageRelay, LOW); // Allumer
    }
    else if (tempActuelle > (TEMP_CIBLE + 0.5)) {
        digitalWrite(chauffageRelay, HIGH); // Éteindre
    }
    delay(10000); // Vérification toutes les 10 secondes
}
```

10.10 Les prises radiocommandées (433 MHz)

Si l'on ne veut pas manipuler les fils 220V par mesure de sécurité (fortement recommandé pour les étudiants!), la solution est d'utiliser des prises télécommandées du commerce.

- On ajoute un petit **émetteur RF 433 MHz** (Radio Fréquence) à l'Arduino.
- On utilise la bibliothèque `RCSwitch` pour simuler le signal de la télécommande d'origine.
- **Avantage** : L'Arduino reste totalement isolé du réseau électrique, et on peut contrôler les lampes à travers les murs (contrairement à l'infrarouge).

Résumé du Chapitre 10

- **Infrarouge (IR)** : Excellente technologie bon marché pour la communication en ligne de mire. La bibliothèque `IRremote` fait tout le travail de décodage.
- **Optocoupleurs** : À utiliser pour déclencher des appareils fonctionnant sur piles (appareils photos, jouets) sans mélanger les circuits.
- **Relais et Sécurité** : Un module relais permet de domotiser un appareil secteur. Ne manipulez le 220V que débranché, et enfermez toujours vos circuits haute tension dans des boîtiers plastiques isolants.
- **L'Alternative RF** : Pour l'enseignement, préférez le "hacking" de prises radiocommandées (433MHz) pour éviter les risques d'électrocution.

Fin du Chapitre 10. Le Chapitre 11 sera consacré aux protocoles de communication matérielle avancés (I2C et SPI).